MODELAGEM DE PROCESSOS E NEGÓCIOS

AULA 1

Prof.ª Adriana Bastos da Costa

CONVERSA INICIAL

Em nosso estudo, vamos discutir alguns conceitos importantes que ajudam a entender como funcionam as empresas. Entender o funcionamento das empresas é fundamental para definir um software que atenda às necessidades do negócio e dos usuários, de maneira completa e correta.

Para entender bem esses conceitos, nesta abordagem vamos discutir e compreender os conceitos de modelagem, processos e negócios.

Modelagem é um termo que pode ter diferentes significados dependendo do contexto em que é utilizado. No geral, ele se refere ao processo de criar representações ou modelos de algo, seja um objeto físico, um sistema abstrato, ou um conjunto de dados.

Quando falamos em TI, a modelagem envolve a criação de modelos conceituais, diagramas ou especificações de software para descrever sua estrutura, comportamento e interações. Ou seja, de maneira bem geral, modelagem é essencialmente o processo de representar algo de interesse de uma maneira que possa ser compreendida, analisada ou manipulada de forma mais fácil e eficaz.

Processos são sequências de atividades inter-relacionadas que transformam entradas em saídas com o objetivo de alcançar um determinado resultado ou objetivo. Eles são fundamentais em muitos campos, desde negócios e engenharia até ciência e natureza. Exemplos de processos incluem o processo de fabricação em uma fábrica, o processo de atendimento ao cliente em uma empresa de serviços, o processo de aprendizado em uma sala de aula, o processo de desenvolvimento de software em uma empresa de tecnologia, entre muitos outros. Os processos são uma parte essencial de como o trabalho é realizado e como os resultados são alcançados em uma ampla variedade de contextos.

Negócios é um termo amplo que se refere às atividades comerciais, industriais ou profissionais realizadas com o objetivo de obter lucro. Em última instância, toda empresa buscar obter lucro em seus negócios. Claro que não estamos falando aqui das empresas sem fins lucrativos, que não são o foco do nosso estudo, apesar dos conceitos estudados poderem ser aplicados também a esse tipo de empresa.

Os negócios podem abranger uma variedade de atividades, incluindo produção, compra e venda de bens ou serviços, investimentos financeiros, prestação de serviços profissionais, entre outros. Logo, os negócios são atividades realizadas com o objetivo de gerar lucro, que abrangem a produção e venda de bens, a prestação de serviços, a gestão de organizações e a interação com mercados e concorrentes.

Pelo que vimos até o momento, este estudo é muito importante para formar bons analistas de sistemas, que tenham a visão sobre o funcionamento dos processos de negócio das empresas, de forma a garantir que o software a ser construído seja o software correto para atender às necessidades do cliente.

Esta abordagem será dividida em 5 tópicos, que estão relacionados entre si e ajudarão a entender os conceitos propostos, sendo eles:

- **Tema 1** – Fundamentos de modelagem de processos (definição de processos de negócios e a importância para o desenvolvimento de software);
- **Tema 2** – Modelagem de processos x modelagem de negócios (BPMN e UML);
- **Tema 3** – Integração da modelagem no ciclo de vida de software e os benefícios da modelagem para a engenharia de software;
- **Tema 4** – Ferramentas atuais de modelagem e Escolha da ferramenta adequada para os casos de uso;
- **Tema 5** – Modelagem para a elaboração de caso de uso e a importância dos conceitos de negócios para a engenharia de software.

TEMA 1 – FUNDAMENTOS DE MODELAGEM DE PROCESSOS

Um modelo de processo é uma representação abstrata e simplificada de um processo de negócio ou atividade organizacional. Ele descreve as etapas sequenciais, atividades, interações e decisões envolvidas na realização de um determinado processo.

Esses modelos podem ser visualizados de várias formas, incluindo fluxogramas, diagramas de atividades, mapas de processo e outros tipos de representações gráficas.

Os modelos de processo são ferramentas essenciais na modelagem de processos de negócios, pois ajudam a documentar e comunicar como o trabalho

é realizado dentro de uma organização. Eles fornecem uma visão geral do processo, permitindo que as partes interessadas entendam melhor sua estrutura, sequência de atividades e interações entre as diferentes partes envolvidas.

Além disso, os modelos de processo são frequentemente utilizados como base para análise e melhoria de processos. Eles podem ajudar a identificar áreas de ineficiência, redundância ou oportunidades de otimização, facilitando a implementação de mudanças que levem a um desempenho organizacional mais eficaz e eficiente.

Portanto, um modelo de processo é uma representação visual que ajuda a compreender, analisar e melhorar os processos de negócios de uma organização. Ele serve como uma ferramenta poderosa para documentar, comunicar e otimizar a forma como o trabalho é realizado dentro da empresa

1.1 O que são processos de negócios

Toda empresa é organizada em processos, que se referem a uma série de atividades inter-relacionadas realizadas por uma organização para alcançar um objetivo específico. Um processo de negócio geralmente tem início, meio e fim e pode envolver múltiplos departamentos ou áreas funcionais dentro da organização.

Todo processo de negócio recebe dados de entrada, processa esses dados e gera um resultado, que é o objetivo do processo em si.

Um processo de negócio representa a forma como a empresa quer gerar seus resultados, é o reflexo da sua organização e da sua forma de trabalhar. Todo processo, portanto, possui algumas características importantes para seu funcionamento adequado, sendo elas:

- **Sequência de atividades**: os processos consistem em uma série ordenada de atividades ou etapas. Cada atividade geralmente contribui para a transformação das entradas em saídas.
- **Transformação ou processamento**: os processos envolvem a transformação de entradas em saídas. Isso pode incluir materiais, informações, energia ou até mesmo ideias.
- **Inter-relacionamento**: as atividades dentro de um processo muitas vezes dependem umas das outras. O resultado de uma atividade pode afetar a próxima, e assim por diante.

- **Objetivo**: os processos são executados com um propósito específico em mente. Eles são projetados para alcançar um determinado resultado ou objetivo, como produzir um produto, fornecer um serviço ou atingir uma meta de negócios.

Para deixar bem claro o que é um processo de negócio, vamos detalhar um processo de atendimento ao cliente em uma loja de varejo. Esse processo envolve várias etapas que são comuns em muitos negócios de varejo, tais como:

- **Recepção do cliente**: o processo começa quando um cliente entra na loja. O primeiro passo é cumprimentar o cliente e oferecer assistência, se necessário. Isso pode envolver um funcionário perguntando se o cliente precisa de ajuda ou apenas oferecendo uma saudação amigável.
- **Identificação das necessidades do cliente**: o próximo passo é entender as necessidades do cliente. Um funcionário pode perguntar ao cliente se ele está procurando algo específico, precisa de ajuda para encontrar um produto ou tem alguma dúvida sobre os produtos disponíveis.
- **Assistência na seleção de produtos**: com base nas necessidades do cliente, o funcionário auxilia na seleção dos produtos adequados. Isso pode envolver explicar as características dos produtos, fornecer recomendações ou sugerir produtos complementares.
- **Processo de compra**: depois que o cliente seleciona os produtos que deseja comprar, ele se dirige ao caixa para fazer o pagamento. O funcionário registra os produtos, calcula o total da compra e processa o pagamento do cliente.
- **Finalização da transação**: após o pagamento, o funcionário conclui a transação. Isso pode incluir fornecer um recibo ao cliente, embalar os produtos, agradecer ao cliente pela compra e convidá-lo a retornar à loja no futuro.
- **Pós-venda**: após a conclusão da transação, o processo pode incluir algumas etapas de pós-venda, como oferecer assistência adicional ao cliente, fornecer informações sobre a política de devolução ou solicitar feedback sobre a experiência de compra.

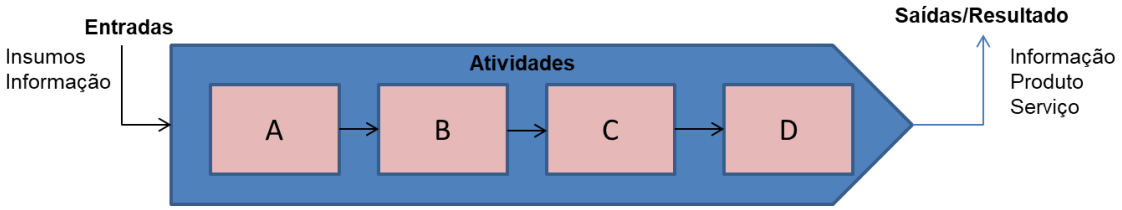
Este é um exemplo simplificado de um processo de negócio, mas demonstra como os processos são compostos por uma série de etapas inter-relacionadas que visam alcançar um objetivo específico, neste caso, satisfazer

as necessidades do cliente durante o processo de compra em uma loja de varejo. E outro ponto importante, um processo de negócio reflete a forma como a empresa organizou as atividades que compõem esse processo. Um mesmo processo pode ter passos diferentes de empresa para empresa, portanto, não existe uma forma única de representar um processo. Depende do funcionamento da empresa e de como ela quer organizar o seu processo.

Logo, os processos de negócios são representados em formato de fluxo, para facilitar a leitura e o entendimento do processo representado. No fluxo de um processo de negócio estão envolvidos profissionais, departamentos, sistemas, máquinas, matéria-prima e várias outras entidades que permitem que o processo seja executado da melhor maneira possível, de forma a entregar um produto ou serviço que atenda às necessidades dos clientes (Becker; Rosemann; Von Uthmann, 2000).

Dessa forma, processos de negócios podem ser entendidos como um conjunto sequenciado de tarefas que são executadas a partir de insumos de entrada, processando essas entradas e gerando uma saída, que é um produto ou serviço.

Figura 1 – Representação de um fluxo de processo



Fonte: Adriana Bastos da Costa.

1.2 A importância dos processos de negócios para o desenvolvimento de software

É preciso entender que um software é construído para automatizar algum processo do cliente, agregando valor para o negócio. Dessa forma, é fundamental conhecer o funcionamento do processo que será automatizado através do software.

Dessa forma, alguns pontos ajudam a entender a importância dos processos de negócio para o desenvolvimento de software:

- **Alinhamento com as necessidades do negócio**: compreender os processos de negócio permite que os desenvolvedores de software identifiquem as necessidades e requisitos específicos do negócio. Isso garante que o software desenvolvido esteja alinhado com os objetivos e estratégias da organização.
- **Identificação de oportunidades de melhoria**: ao analisar os processos existentes, os desenvolvedores podem identificar áreas em que o software pode automatizar tarefas manuais, simplificar fluxos de trabalho complexos ou melhorar a eficiência operacional. Isso permite que o software não apenas atenda às necessidades atuais, mas também promova a melhoria contínua dos processos de negócio.
- **Definição de requisitos claros**: compreender os processos de negócio ajuda na definição de requisitos claros e específicos para o software. Isso inclui identificar as funcionalidades necessárias, os dados que precisam ser capturados e processados, os usuários finais envolvidos e os casos de uso esperados.
- **Redução de retrabalho e custos**: conhecer os processos de negócio desde o início ajuda a evitar retrabalho e custos adicionais associados a mudanças de escopo ou requisitos mal definidos. Ao entender completamente as necessidades do negócio, os desenvolvedores podem construir um software que atenda às expectativas desde o início.
- **Maximização do valor para o cliente**: desenvolver um software que esteja alinhado com os processos de negócio permite que as organizações ofereçam mais valor aos clientes. Isso pode incluir melhorias na experiência do usuário, entrega mais rápida de produtos ou serviços e maior eficiência operacional.

Dessa forma, conhecer os processos de negócio antes de construir um software é crucial para garantir que o software atenda às necessidades e expectativas do negócio, promovendo eficiência, redução de custos e maximização do valor para o cliente.

Quanto melhor conhecido for o funcionamento dos processos a serem automatizados por meio de um software, melhor será o resultado gerado e, consequentemente, a satisfação do cliente.

Uma vez conhecido o seu funcionamento, é preciso modelar o processo para garantir o entendimento completo de todas as atividades e as dependências

de execução de cada uma. A modelagem permite visualizar, por meio de um fluxo, o funcionamento do processo, passo a passo. Vamos detalhar melhor a modelagem nos tópicos a seguir.

TEMA 2 – MODELAGEM DE PROCESSOS X MODELAGEM DE NEGÓCIOS

Já entendemos que a modelagem é uma ferramenta que nos ajuda a compreender o funcionamento de um processo, entendendo o passo a passo das atividades que devem ser executadas, assim como a ordem de execução de cada uma das atividades.

Envolvendo modelagem, existem dois termos que se complementam, a *modelagem de processo* e a *modelagem de negócios*. Vamos entender a diferença e os pontos em comum entre os dois termos:

- **Modelagem de Negócios**: é uma atividade que visa descrever e entender como uma organização funciona, incluindo sua estrutura, processos, produtos, serviços e interações com clientes e outras partes interessadas. A modelagem de negócios pode envolver a criação de diversos artefatos, como modelos de processos, modelos de dados, mapas de valor, entre outros.
- **Modelagem de Processos**: é o processo de examinar os processos de uma organização para identificar pontos fortes, pontos fracos, ineficiências e oportunidades de melhoria. Isso pode envolver a coleta e análise de dados, a realização de entrevistas com stakeholders, a observação direta de processos em ação e a aplicação de técnicas de modelagem para representar os processos existentes e propostos.

Portanto, a modelagem de processos e a modelagem de negócios são duas abordagens distintas, mas relacionadas, que têm como objetivo entender e representar aspectos diferentes de uma organização. Detalhando um pouco mais cada um dos termos:

- **Modelagem de Processos:** a modelagem de processos concentra-se na representação detalhada e sequencial das atividades, tarefas e interações que ocorrem dentro de um processo específico. O objetivo principal é entender como o trabalho é realizado em uma organização, identificando os passos necessários para realizar uma determinada função ou alcançar um determinado objetivo. As técnicas comuns de modelagem de

processos incluem diagramas de fluxo de processos, fluxogramas e descrições textuais detalhadas. A modelagem de processos é frequentemente usada para analisar e otimizar processos existentes, identificar gargalos, eliminar atividades redundantes e melhorar a eficiência operacional.

- **Modelagem de Negócios:** a modelagem de negócios é uma abordagem mais abrangente que se concentra na representação holística de uma organização, incluindo seus processos, estrutura, estratégias, recursos e relacionamentos. O objetivo é entender como a organização opera como um todo, incluindo suas atividades, fluxos de valor, relacionamentos com clientes e partes interessadas, e como ela cria valor. As técnicas comuns de modelagem de negócios incluem modelos de negócios canvas, mapas de valor, diagramas de contexto e diagramas de stakeholders. A modelagem de negócios é usada para analisar e visualizar todos os aspectos de uma organização, ajudando a identificar oportunidades de inovação, definir estratégias de negócios, entender o posicionamento no mercado e comunicar de forma eficaz a visão e os objetivos da organização.

Logo, enquanto a modelagem de processos se concentra na representação detalhada e sequencial das atividades dentro de um processo específico, a modelagem de negócios se concentra na representação holística da organização como um todo, incluindo seus processos, estrutura, estratégias e relacionamentos.

Ambas as abordagens são importantes para entender e melhorar uma organização, e muitas vezes são usadas em conjunto para uma compreensão mais completa e integrada do seu funcionamento. No contexto de desenvolvimento de software, ambas as abordagens agregam valor no entendimento do que deverá ser construído, ajudando na modelagem do melhor software para a empresa.

2.1 O que é BPM

BPM significa *Business Process Management*, que em português pode ser traduzido como Gerenciamento de Processos de Negócio. É uma abordagem metodológica para identificar, desenhar, executar, documentar, medir, monitorar

e controlar processos de negócio, visando melhorar continuamente a eficiência e eficácia das operações organizacionais.

O BPM envolve o uso de diversas técnicas, ferramentas e tecnologias para analisar, modelar e otimizar os processos de negócio. Algumas das atividades comuns associadas ao BPM incluem:

- **Identificação de processos críticos**: identificar os processos de negócio que são fundamentais para o sucesso da organização e para atingir seus objetivos estratégicos.
- **Modelagem de processos**: representar graficamente os processos de negócio usando técnicas como diagramas de fluxo de processos, permitindo uma compreensão visual das atividades, interações e fluxos de trabalho.
- **Análise e otimização de processos**: analisar os processos existentes para identificar gargalos, atividades redundantes, pontos de melhoria e oportunidades de automação. Em seguida, otimizar os processos para melhorar a eficiência, reduzir custos e tempo de execução, e aumentar a qualidade e a consistência das operações.
- **Automação de processos**: automatizar tarefas e atividades por meio de sistemas de software, workflows automatizados e integrações de sistemas para agilizar a execução de processos, reduzir erros e aumentar a produtividade.
- **Monitoramento e controle de processos**: implementar sistemas de monitoramento e controle para acompanhar o desempenho dos processos em tempo real, identificar desvios e tomar medidas corretivas quando necessário.
- **Gestão de mudanças**: gerenciar e facilitar a implementação de mudanças nos processos de negócio, garantindo que as mudanças sejam planejadas, comunicadas, testadas e adotadas de forma eficaz.

O BPM é uma abordagem holística que visa melhorar o desempenho organizacional, promover a inovação e a adaptação às mudanças do mercado, e aumentar a satisfação do cliente ao alinhar os processos de negócio com os objetivos estratégicos da organização.

2.2 O que é UML

UML, ou *Unified Modeling Language* (Linguagem de Modelagem Unificada), é uma linguagem visual padrão usada para modelar sistemas de software. Ela fornece um conjunto de notações gráficas e regras de sintaxe para representar diferentes aspectos de sistemas de software, incluindo estrutura, comportamento, interações e arquitetura. É uma linguagem usada para modelagem de software, independentemente da metodologia de desenvolvimento de software utilizada. O foco aqui é criar uma linguagem padrão, conhecida e entendida pelos envolvidos na construção do projeto, de forma a deixar claro o funcionamento do fluxo do processo a ser automatizado.

A UML é amplamente utilizada na engenharia de software para auxiliar na concepção, visualização, documentação e comunicação de sistemas complexos. Ela permite aos desenvolvedores, analistas e stakeholders descrever e entender os sistemas de uma maneira mais clara e precisa. Alguns dos diagramas da UML mais comuns:

- **Diagrama de Caso de Uso**: descreve os diferentes casos de uso de um sistema, mostrando como os usuários interagem com o sistema e os resultados esperados.
- **Diagrama de Classes**: mostra a estrutura estática de um sistema, incluindo classes, atributos, métodos e relacionamentos entre as classes.
- **Diagrama de Sequência**: representa a interação entre os objetos em um determinado cenário de uso, mostrando a ordem das mensagens trocadas entre os objetos.
- **Diagrama de Estado**: descreve os estados possíveis de um objeto e as transições entre esses estados em resposta a eventos.
- **Diagrama de Atividades**: modela o fluxo de controle de um processo ou algoritmo, mostrando as atividades e as decisões tomadas em cada etapa.
- **Diagrama de Componentes e Diagrama de Implantação**: representam a estrutura física de um sistema, mostrando os componentes de software e sua distribuição em hardware físico.

Esses são apenas alguns exemplos dos tipos de diagramas disponíveis na UML. A UML fornece uma linguagem padronizada e amplamente reconhecida para comunicar conceitos e projetos de software, o que a torna uma ferramenta

valiosa para a engenharia de software e o desenvolvimento de sistemas complexos.

Vamos detalhar nos próximos tópicos tanto o BPM quanto a UML, e entender como essas ferramentas podem auxiliar no entendimento e correta modelagem de um software.

TEMA 3 – INTEGRAÇÃO DA MODELAGEM NO CICLO DE VIDA DE SOFTWARE E OS BENEFÍCIOS DA MODELAGEM PARA A ENGENHARIA DE SOFTWARE

A modelagem e o ciclo de vida do software estão intrinsecamente relacionados ao longo de todo o processo de desenvolvimento de um projeto de software. A modelagem é uma atividade essencial em várias fases do ciclo de vida do software e desempenha um papel crucial na compreensão, concepção, implementação e manutenção do sistema de software.

A modelagem é uma ferramenta poderosa que é usada em várias fases do ciclo de vida do software para ajudar a entender, projetar, implementar, testar e manter sistemas de software de forma eficaz e eficiente. Ela desempenha um papel fundamental na comunicação, análise e visualização de sistemas complexos, proporcionando uma base sólida para o desenvolvimento de software bem-sucedido.

A modelagem desempenha um papel fundamental na engenharia de software, proporcionando uma série de benefícios que contribuem para o sucesso dos projetos de desenvolvimento de software. Alguns dos principais benefícios da modelagem para a engenharia de software são:

- **Comunicação clara e eficaz:** a modelagem fornece uma linguagem visual e padronizada para comunicar conceitos e ideias entre membros da equipe, partes interessadas e clientes. Isso ajuda a garantir que todos tenham uma compreensão comum dos requisitos, design e funcionalidades do sistema.
- **Compreensão abrangente do sistema:** a criação de modelos permite uma análise detalhada e uma compreensão profunda do sistema antes mesmo de começar a codificação. Isso ajuda a identificar potenciais problemas, desafios e oportunidades de melhoria desde o início do processo de desenvolvimento.

- **Identificação antecipada de requisitos:** a modelagem ajuda na identificação antecipada dos requisitos do sistema, permitindo que sejam capturados de forma clara e precisa. Isso reduz o risco de interpretações erradas ou mal-entendidos dos requisitos, o que pode levar a retrabalho e atrasos no projeto.
- **Facilitação da tomada de decisões:** modelos visualmente representados fornecem uma base sólida para a tomada de decisões durante o processo de desenvolvimento. Eles permitem avaliar diferentes possibilidades e tomar decisões baseadas em informação que melhor atendam aos objetivos do projeto.
- **Detecção precoce de problemas e defeitos**: a modelagem ajuda a identificar problemas de design, inconsistências ou lacunas nos requisitos antes que eles se tornem problemas mais sérios durante a implementação. Isso economiza tempo e recursos, evitando retrabalho e garantindo a qualidade do software desde o início.
- **Facilitação da manutenção e evolução do sistema**: modelos bem documentados facilitam a manutenção e a evolução contínua do sistema ao longo do tempo. Eles fornecem uma referência clara para entender a estrutura e o comportamento do sistema, ajudando a realizar mudanças de forma eficiente e eficaz.
- **Promoção da reutilização de artefatos**: modelos podem ser reutilizados em diferentes fases do ciclo de vida do software e em projetos subsequentes. Isso aumenta a eficiência do desenvolvimento, reduzindo o esforço necessário para criar novos artefatos e acelerando o tempo de entrega do projeto.

No geral, a modelagem desempenha um papel crucial na criação de software de alta qualidade, facilitando a comunicação, a compreensão, a tomada de decisões e a manutenção ao longo do ciclo de vida do projeto.

3.1 Ciclo de vida de desenvolvimento de software

Desenvolver um software não é algo simples e nem trivial, por isso mesmo o processo de desenvolvimento de software precisa ser organizado em etapas, de forma a garantir que as partes menores sejam mais bem gerenciadas e

ajudem na construção do todo. Essa organização em etapas é chamada de c*iclo de vida de desenvolvimento de software*.

O ciclo de vida do software é uma representação do processo pelo qual um sistema de software é concebido, desenvolvido, mantido e eventualmente aposentado. Ele descreve as fases pelas quais o software passa desde sua concepção até sua descontinuação e inclui todas as atividades relacionadas ao desenvolvimento, implantação, operação e manutenção do software ao longo do tempo.

Existem duas abordagens bastante conhecidas e utilizadas atualmente para a construção de software. Estamos falando das abordagens tradicional e ágil de desenvolvimento de software.

Cada uma está organizada seguindo um ciclo de vida diferente. Vamos analisar dois ciclos de vida conhecidos pelas equipes de desenvolvimento de software. Para a abordagem tradicional, vamos analisar o ciclo de vida que organiza o modelo cascata. E para a abordagem ágil, vamos analisar o ciclo de vida do Scrum.

3.1.1 Metodologia Cascata de desenvolvimento de software:

- **Análise de Requisitos:** nesta fase, os requisitos do sistema são coletados, analisados e documentados. Isso inclui identificar as necessidades dos usuários, entender o contexto do sistema e definir as funcionalidades que o software deve oferecer.
- **Projeto:** na fase de projeto, os requisitos identificados na fase anterior são traduzidos em uma arquitetura de software e um design detalhado. Isso inclui a definição de componentes, interfaces, estrutura de dados e algoritmos.
- **Implementação:** durante a fase de implementação, o software é codificado com base no design estabelecido. Isso envolve escrever código-fonte, criar bancos de dados, desenvolver interfaces de usuário e integrar componentes.
- **Teste:** na fase de teste, o software é submetido a uma série de testes para verificar se ele atende aos requisitos especificados, identificar defeitos e garantir sua correção antes da entrega.

- **Implantação:** uma vez que o software tenha passado nos testes e esteja pronto para uso, ele é implantado em ambiente de produção e disponibilizado para os usuários finais.
- **Manutenção:** após a implantação, o software entra na fase de manutenção, na qual é sujeito a atividades de correção de bugs, implementação de melhorias, atualizações de segurança e suporte ao usuário ao longo do tempo.

3.1.2 Metodologia Scrum de desenvolvimento de software:

O Scrum, também chamado de fra*mework ágil de desenvolvimento de software*, é uma metodologia que organiza o ciclo de vida do projeto em uma série de eventos, papéis e artefatos. Vamos detalhar como o ciclo de vida do Scrum é organizado:

- **Eventos do Scrum**:
  - **Sprint**: o Sprint é o período de tempo fixo durante o qual o trabalho é realizado. Geralmente, tem duração de 2 a 4 semanas. Cada Sprint começa com uma reunião de planejamento do Sprint e termina com uma revisão do Sprint e uma retrospectiva do Sprint.
  - **Reunião de Planejamento do Sprint**: no início de cada Sprint, a equipe Scrum se reúne para definir quais itens do Backlog do Produto serão trabalhados durante o Sprint e como eles serão implementados. Essa reunião resulta no Sprint Backlog, que é a lista de itens selecionados para o Sprint e as tarefas necessárias para completá-los.
  - **Daily Scrum (Scrum Diário)**: esta é uma reunião diária de curta duração (normalmente 15 minutos) em que a equipe Scrum se reúne para sincronizar o progresso do trabalho, discutir o que foi feito desde a última reunião, o que será feito até a próxima reunião e quaisquer obstáculos que estejam impedindo o progresso.
  - **Revisão do Sprint**: no final de cada Sprint, a equipe Scrum apresenta o trabalho concluído ao *Product Owner* e a outras partes interessadas para revisão e feedback. Esta é uma oportunidade para discutir o que foi realizado durante o Sprint e obter feedback sobre o produto entregue.
  - **Retrospectiva do Sprint**: após a Revisão do Sprint, a equipe Scrum realiza uma retrospectiva para refletir sobre o Sprint e identificar melhorias. Eles discutem o que funcionou bem, o que poderia ser

melhorado e quais ações serão tomadas para melhorar o processo no próximo Sprint.

- **Papéis do Scrum**:
  - ***Product Owner* (PO)**: é responsável por maximizar o valor do produto e pela gestão do Backlog do Produto, priorizando itens, definindo requisitos e aceitando entregas.
  - ***Scrum Master***: é responsável por garantir que a equipe Scrum entenda e siga os princípios e práticas do Scrum, removendo obstáculos que possam impedir o progresso e facilitando os eventos do Scrum.
  - **Equipe Scrum**: é composta por profissionais multifuncionais que realizam o trabalho necessário para entregar um incremento de produto ao final de cada Sprint.
- **Artefatos do Scrum**:
  - **Backlog do Produto**: é uma lista de todos os itens que podem ser necessários para o produto, incluindo novas funcionalidades, correções de bugs, melhorias, etc. É de responsabilidade do PO priorizar e gerenciar o Backlog do Produto.
  - **Sprint Backlog**: é uma lista de todos os itens do Backlog do Produto selecionados para o Sprint atual, juntamente com as tarefas necessárias para completá-los. É de responsabilidade da equipe Scrum gerenciar o Sprint Backlog durante o Sprint.
  - **Incremento do Produto**: é o resultado do trabalho realizado durante o Sprint, que deve ser funcional e potencialmente entregável. O objetivo de cada Sprint é produzir um incremento de produto que adicione valor ao produto geral.

Esses são os principais elementos que compõem o ciclo de vida do Scrum. O Scrum é um framework iterativo e incremental, o que significa que os eventos, papéis e artefatos são repetidos em cada Sprint, permitindo entregas rápidas e frequentes de valor ao cliente.

Como ficou claro pelos exemplos analisados acima, dependendo da abordagem escolhida, a forma de organizar o desenvolvimento de software muda. Mas o importante é entender que o objetivo principal do ciclo de vida é dividir algo grande e complexo em partes menores e mais gerenciáveis. Dessa forma, independentemente da abordagem adotada, o ciclo de vida do software é essencial para garantir o desenvolvimento, operação e manutenção bem-

sucedidos do software ao longo do tempo, proporcionando um guia estruturado para o processo de desenvolvimento de software.

3.2 O que é Engenharia de Software

A engenharia de software se concentra na aplicação de princípios e práticas de engenharia para o desenvolvimento de software. Ela envolve a aplicação de métodos sistemáticos, técnicas e ferramentas para criar, testar, manter e gerenciar software de alta qualidade de forma eficiente e eficaz. Ou seja, como desenvolver um software é algo complexo, é preciso criar processos que organizem todo o trabalho que deve ser realizado, para atingir os resultados esperados. É preciso planejar tecnicamente um software, antes dele ser realmente construído. É por isso que utilizamos a engenharia de software no processo de construção de um produto de software.

A engenharia de software abrange várias áreas e atividades ao longo do ciclo de vida do desenvolvimento de software. Ela envolve o uso de metodologias de desenvolvimento de software para organizar e gerenciar o processo de desenvolvimento. Também inclui o uso de ferramentas de desenvolvimento, controle de versão, gerenciamento de projetos, testes e depuração. Ou seja, a engenharia de software é a forma como chamamos o processo de desenvolvimento de software, organizado em etapas de um ciclo de vida, conforme visto anteriormente.

O objetivo da engenharia de software é criar softwares de alta qualidade que atenda às necessidades do cliente, que seja robusto, seguro, eficiente e fácil de manter. Isso é alcançado por meio da aplicação de princípios de engenharia, boas práticas de desenvolvimento e colaboração entre equipes multidisciplinares ao longo de todo o ciclo de vida do software.

A engenharia de software desempenha um papel essencial na garantia da qualidade do software, desde a definição de requisitos de qualidade até a implementação de práticas de desenvolvimento e teste que promovam a qualidade em todas as fases do ciclo de vida do desenvolvimento. Ao adotar abordagens e técnicas de engenharia de software orientadas à qualidade, as organizações podem criar software que seja confiável, seguro, eficiente e fácil de manter.

TEMA 4 – FERRAMENTAS ATUAIS DE MODELAGEM E ESCOLHA DA FERRAMENTA ADEQUADA PARA OS CASOS DE USO

Já discutimos que modelar um software antes deste ser codificado é fundamental para garantir que todos os detalhes do processo de negócio foram compreendidos e inseridos na lógica do software.

Portanto, modelar um software é a etapa do ciclo de vida que envolve o entendimento lógico e físico do que deverá ser construído utilizando uma linguagem de programação.

Embora ferramentas de modelagem sejam úteis e muitas vezes acelerem o processo de modelagem de software, elas não são estritamente necessárias. É possível usar abordagens mais simples e baseadas em texto, prototipagem rápida ou modelagem colaborativa, dependendo das necessidades do projeto e das preferências da equipe. O importante é garantir que o modelo seja claro, compreensível e eficaz na comunicação dos conceitos e requisitos do software.

Mas, apesar de não ser mandatório, o uso de uma ferramenta pode garantir uma melhor qualidade no trabalho desenvolvido, uma vez que pode trazer mais organização para a modelagem. Dependendo do tamanho e da complexidade do software, a modelagem pode envolver vários diagramas e várias verificações.

Existem várias ferramentas modernas disponíveis para modelagem de software, cada uma com suas próprias características e funcionalidades. Vamos avaliar algumas das ferramentas mais populares utilizadas atualmente:

- **Lucidchart:**
  - O Lucidchart é uma ferramenta baseada na web que permite criar uma ampla variedade de diagramas, incluindo diagramas UML, diagramas de fluxo de processos, diagramas de rede, entre outros.
  - Funciona com uma interface intuitiva de arrastar e soltar, em que os usuários podem selecionar elementos predefinidos e conectá-los para representar visualmente os diferentes aspectos de um sistema de software.
  - Possui recursos de colaboração em tempo real, o que permite que equipes trabalhem juntas em um mesmo diagrama de forma remota e simultânea.
- **Visual Paradigm**:

- O Visual Paradigm é uma suíte de ferramentas de modelagem que oferece suporte a uma ampla gama de diagramas, incluindo diagramas UML, diagramas de fluxo de processos, diagramas de caso de uso, entre outros.
- Oferece uma variedade de recursos avançados, como geração de código, engenharia reversa, simulação de modelos, e suporte a padrões de modelagem.
- É uma ferramenta mais robusta, adequada para projetos de grande porte e equipes de desenvolvimento.

- **Draw.io**:
  - O Draw.io é uma ferramenta gratuita e de código aberto para criação de diagramas, incluindo diagramas UML, diagramas de fluxo, organogramas, entre outros.
  - Funciona em um ambiente baseado na web e oferece integração com várias plataformas de armazenamento em nuvem, como Google Drive, OneDrive e Dropbox.
  - Possui uma ampla variedade de formas e ícones disponíveis para representar diferentes elementos em um diagrama, além de recursos de formatação e estilização.
- **Enterprise Architect**:
  - O Enterprise Architect é uma ferramenta de modelagem de software mais avançada, amplamente utilizada em ambientes corporativos e industriais.
  - Oferece suporte a uma variedade de padrões e notações de modelagem, incluindo UML, BPMN, SysML e mais.
  - Possui recursos de colaboração em equipe, gerenciamento de requisitos, rastreamento de mudanças e integração com outras ferramentas de desenvolvimento de software.
- **Astah**:
  - O Astah é uma ferramenta de modelagem UML e diagramação de software que oferece uma interface simples e intuitiva.
  - É popular entre desenvolvedores e analistas de negócios devido à sua facilidade de uso e foco na criação de diagramas claros e eficazes.
  - Oferece recursos como geração de código, integração com IDEs e suporte a diferentes tipos de diagramas, como diagramas de classe,

diagramas de sequência e diagramas de caso de uso. Uma IDE é o ambiente de Desenvolvimento Integrado (*Integrated Development Environment*), ou seja, o ambiente onde um software é codificado e compilado para ser executado.

Essas são apenas algumas das ferramentas mais populares e modernas para modelagem de software disponíveis atualmente. Cada uma tem suas próprias características e funcionalidades, então é importante avaliar suas necessidades específicas antes de escolher a ferramenta mais adequada para o seu projeto.

4.1 O que são casos de uso

Uma das primeiras etapas da modelagem de software está relacionada com os requisitos do negócio que devem ser entendidos e modelados para serem traduzidos em linhas de código que vão processar dados de entrada e gerar os resultados esperados.

Dessa forma, para compreender esse início de modelagem, vamos explorar o que são casos de uso. Casos de uso são uma técnica amplamente utilizada na engenharia de requisitos para capturar e descrever os requisitos funcionais de um sistema de software a partir da perspectiva do usuário.

Requisitos funcionais são uma parte fundamental da engenharia de requisitos e descrevem as funcionalidades específicas que um sistema de software deve fornecer para atender às necessidades dos usuários e alcançar os objetivos do negócio. Eles definem o comportamento do sistema, especificando as interações entre o usuário e o sistema, bem como as operações que o sistema deve realizar em resposta a essas interações.

Em termos simples, os requisitos funcionais respondem à pergunta "**o que o sistema deve fazer?**" e são geralmente expressos em termos de ações, operações ou comportamentos observáveis. Eles podem incluir:

- **Funcionalidades do Sistema**: descrevem as operações específicas que o sistema deve ser capaz de realizar. Isso pode incluir funções de processamento de dados, cálculos, manipulação de informações, geração de relatórios, entre outros.
- **Regras de Negócio**: especificam as regras ou políticas que o sistema deve seguir ao realizar determinadas operações. Isso pode incluir

restrições de validação de dados, regras de fluxo de trabalho, políticas de segurança, entre outros.

É fundamental que os requisitos funcionais sejam claros, específicos, completos e verificáveis para garantir que o sistema seja desenvolvido de acordo com as expectativas dos usuários. Eles geralmente são documentados em uma especificação de requisitos funcionais, que serve como referência para o design, implementação e teste do sistema de software. Além disso, os requisitos funcionais podem evoluir ao longo do tempo à medida que novos requisitos são identificados ou os requisitos existentes são modificados para atender às mudanças nas necessidades do negócio.

Os casos de uso representam situações ou interações específicas entre o sistema e seus usuários, descrevendo como o sistema será utilizado para alcançar determinados objetivos ou realizar determinadas tarefas. Ou seja, os casos de uso descrevem os requisitos funcionais de um sistema de software. Os casos de uso são compostos por duas partes principais:

- **Ator(es):** representam os usuários ou sistemas externos que interagem com o sistema em questão. Os atores podem ser pessoas, outros sistemas, dispositivos de hardware, entre outros. Cada ator é identificado e nomeado para indicar seu papel específico no contexto do sistema.
- **Cenário Principal:** descreve a sequência de eventos que ocorrem quando um ator interage com o sistema para alcançar um objetivo específico. O cenário principal geralmente é descrito em uma série de passos ou ações, mostrando como o sistema responde às entradas do usuário e executa as operações necessárias para satisfazer o objetivo do caso de uso.

Além do cenário principal, um caso de uso também pode incluir:

- **Cenários Alternativos:** descrevem variações ou exceções ao cenário principal que podem ocorrer durante a execução do caso de uso. Isso pode incluir condições de erro, caminhos alternativos ou decisões tomadas pelo sistema ou pelo usuário.
- **Pré-condições e Pós-condições:** especificam as condições que devem ser verdadeiras antes e depois da execução do caso de uso, respectivamente. As pré-condições descrevem o estado inicial do sistema antes que o caso de uso possa ser iniciado, enquanto as pós-condições

descrevem o estado resultante do sistema após a conclusão do caso de uso.

Os casos de uso são frequentemente representados visualmente por meio de diagramas de casos de uso, que mostram os atores, os casos de uso e as relações entre eles. Esses diagramas fornecem uma visão geral do sistema a partir da perspectiva do usuário, ajudando a identificar os principais recursos e funcionalidades que o sistema deve oferecer.

Fica claro, então, que os casos de uso são uma ferramenta valiosa para capturar e documentar os requisitos funcionais de um sistema de software, fornecendo uma base sólida para o design, desenvolvimento e teste do sistema.

No próximo tópico, vamos discutir um pouco mais como modelar casos de uso.

4.2 Como escolher a ferramenta adequada para modelar os casos de uso

Vimos que existem várias ferramentas para modelagem de software. Mas como podemos decidir entre uma ferramenta ou outra?

Pois bem, escolher a ferramenta certa para modelagem de software pode depender de vários fatores, incluindo as necessidades do projeto, as preferências da equipe e o orçamento disponível. Vamos listar aqui algumas considerações importantes a serem levadas em conta ao escolher uma ferramenta para modelagem de software:

- **Requisitos do Projeto**: avalie os requisitos específicos do seu projeto de desenvolvimento de software. Determine quais tipos de diagramas ou modelos precisam ser criados e quais recursos são essenciais para o sucesso do projeto.
- **Compatibilidade de Linguagem e Plataforma**: verifique se a ferramenta oferece suporte à linguagem de programação e às plataformas de desenvolvimento que serão usadas no projeto. Além disso, considere se a ferramenta é compatível com os sistemas operacionais que sua equipe utiliza.
- **Facilidade de Uso**: escolha uma ferramenta que seja fácil de aprender e usar, especialmente se sua equipe não tiver experiência prévia com modelagem de software. Uma interface intuitiva e recursos de arrastar e soltar podem facilitar o processo de modelagem.

- **Flexibilidade e Personalização**: procure por uma ferramenta que ofereça flexibilidade e opções de personalização para atender às necessidades específicas do seu projeto. Isso pode incluir a capacidade de criar modelos personalizados, definir propriedades e estilos, e integrar com outras ferramentas de desenvolvimento.
- **Colaboração em Equipe**: se sua equipe trabalha de forma colaborativa, escolha uma ferramenta que ofereça recursos de colaboração em tempo real, como edição simultânea, comentários e compartilhamento de modelos. Isso facilitará a comunicação e a colaboração entre os membros da equipe.
- **Recursos Avançados**: considere se a ferramenta oferece recursos avançados que podem ser úteis para o seu projeto, como geração de código, engenharia reversa, simulação de modelos, análise de impacto e integração com outras ferramentas de desenvolvimento.
- **Suporte e Comunidade**: verifique se a ferramenta tem uma comunidade ativa de usuários e desenvolvedores, bem como suporte técnico disponível caso você precise de assistência ou enfrentar problemas durante o uso da ferramenta.
- **Custo e Licenciamento**: por fim, leve em consideração o custo e o modelo de licenciamento da ferramenta. Algumas ferramentas são gratuitas ou de código aberto, enquanto outras requerem uma assinatura paga ou licenciamento por usuário. Escolha uma opção que se encaixe no seu orçamento e que ofereça um bom retorno sobre o investimento.

É preciso avaliar cada ferramenta de forma detalhada, para que a escolha da ferramenta ajude no processo de modelagem e facilite o trabalho de toda a equipe do desenvolvimento do software. A ferramenta tem como objetivo melhorar a produtividade e melhorar o trabalho da equipe.

TEMA 5 – MODELAGEM PARA A ELABORAÇÃO DE CASO DE USO E A IMPORTÂNCIA DOS CONCEITOS DE NEGÓCIOS PARA A ENGENHARIA DE SOFTWARE

Já discutimos a importância do entendimento do processo de negócio para a correta construção de um software. A representação do processo de

negócio por meio de um fluxo permite que todos os envolvidos no projeto consigam entender o funcionamento do processo.

O fluxo do processo é composto por requisitos funcionais, que são descritos em formato de casos de uso, correto?

Dessa forma, a modelagem de casos de uso desempenha um papel crucial na construção de um software por várias razões. Vamos explorar algumas dessas principais razões, reforçando os conceitos já discutimos até aqui:

- **Compreensão das Necessidades dos Usuários**: os casos de uso ajudam a equipe de desenvolvimento a entender as necessidades e expectativas dos usuários finais do software. Eles fornecem uma representação clara das funcionalidades e interações que os usuários desejam realizar com o sistema, permitindo que os desenvolvedores priorizem e projetem adequadamente os recursos do software.
- **Especificação de Requisitos Funcionais**: os casos de uso servem como uma maneira eficaz de capturar e documentar os requisitos funcionais do software. Eles descrevem o comportamento esperado do sistema em resposta às entradas dos usuários, ajudando a garantir que os desenvolvedores construam o sistema de acordo com as expectativas dos stakeholders.
- **Comunicação Efetiva entre as Partes Interessadas**: os casos de uso fornecem uma linguagem comum para a comunicação entre as partes interessadas, incluindo desenvolvedores, clientes, usuários finais e outros stakeholders. Eles ajudam a garantir que todas as partes envolvidas tenham uma compreensão clara das funcionalidades e comportamentos do sistema, reduzindo assim o risco de mal-entendidos e ambiguidades.
- **Guiar o Design e o Desenvolvimento**: os casos de uso servem como uma base para o design e o desenvolvimento do software. Eles ajudam os desenvolvedores a identificar os principais recursos e funcionalidades do sistema, bem como a definir a arquitetura e a estrutura do software para atender aos requisitos dos casos de uso.
- **Avaliação e Validação do Software**: os casos de uso podem ser usados para avaliar e validar o software durante o desenvolvimento e os testes. Eles fornecem critérios claros para determinar se o software atende aos requisitos dos usuários e se está funcionando conforme o esperado.

- **Facilitar a Manutenção e Evolução do Software**: uma vez construído, os casos de uso podem servir como uma referência para a manutenção e evolução contínua do software. Eles ajudam a garantir que as mudanças e atualizações feitas no software sejam alinhadas com as necessidades e expectativas dos usuários.

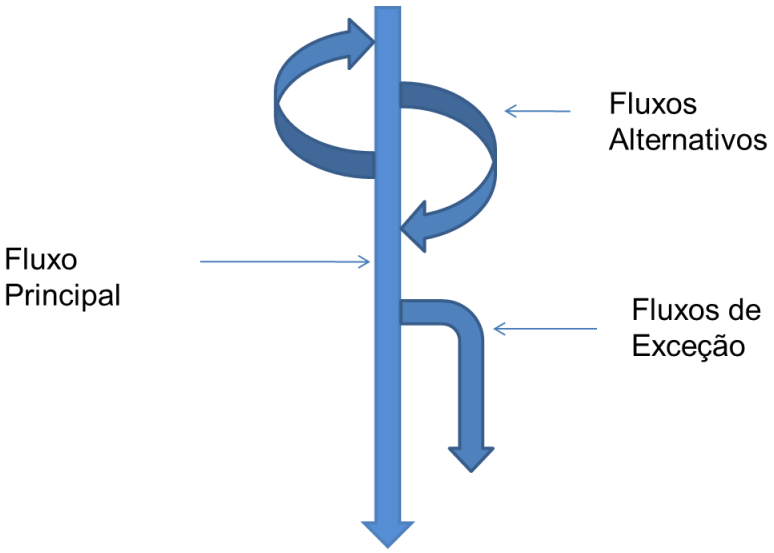
Logo, a modelagem de casos de uso é uma prática essencial na construção de software, pois ajuda a garantir que o software atenda às necessidades dos usuários e dos stakeholders, seja bem projetado e desenvolvido, e possa evoluir de forma eficaz ao longo do tempo.

5.1 Elaboração de casos de uso

É importante entender que documentar casos de uso significa descrever como o software deve se comportar, pois o caso de uso descreve os requisitos funcionais de um software. Logo, em última instância, documentar um caso de uso significa descrever fluxos de eventos, listando todos os passos seguidos para se chegar a um resultado.

Os fluxos de eventos em um caso de uso mostram o que deve ser executado em cada funcionalidade do software, sendo que existem três tipos de fluxos:

Figura 2 – Fluxos de Eventos em um Caso de Uso



Fonte: Adriana Bastos da Costa.

Modelar casos de uso é uma prática importante na engenharia de requisitos para capturar e descrever as interações entre os usuários e o sistema de software, correto? Pois bem, para deixar bem claro esse trabalho, vamos listar os passos básicos para modelar casos de uso:

- **Identificar os atores**: comece identificando os atores principais que interagem com o sistema. Um ator pode ser qualquer entidade externa que interage com o sistema, como um usuário humano, outro sistema, dispositivo de hardware, etc.
- **Identificar os casos de uso**: identifique as diferentes funcionalidades ou tarefas que o sistema deve realizar para atender às necessidades dos usuários. Cada caso de uso deve representar uma ação ou interação específica que um usuário pode realizar com o sistema para alcançar um objetivo.
- **Descrever os casos de uso**: para cada caso de uso identificado, descreva-o em detalhes, incluindo seu nome, descrição, pré-condições, pós-condições e cenário principal de execução. Isso envolve descrever os passos que um usuário seguirá ao interagir com o sistema para realizar a tarefa específica.
- **Identificar cenários alternativos**: além do cenário principal, identifique quaisquer cenários alternativos ou exceções que possam ocorrer durante a execução do caso de uso. Isso pode incluir condições de erro, caminhos alternativos ou decisões tomadas pelo usuário ou pelo sistema.
- **Criar diagramas de casos de uso**: use diagramas de casos de uso para visualizar e organizar os casos de uso identificados e as relações entre eles. No diagrama, cada caso de uso é representado como um oval e os atores são representados como caixas externas. As linhas de comunicação conectam os atores aos casos de uso que eles iniciam.
- **Validar e revisar**: após modelar os casos de uso, revise-os com os stakeholders e membros da equipe para garantir que estejam completos, consistentes e compreensíveis. Faça os ajustes necessários com base no feedback recebido.

É importante lembrar que os casos de uso devem ser focados nos objetivos e necessidades dos usuários, em vez de detalhes técnicos de implementação. Os detalhes técnicos serão acrescentados em outro momento

do ciclo de vida do software. Eles devem descrever o comportamento observável do sistema do ponto de vista do usuário, ajudando a garantir que o sistema desenvolvido atenda às expectativas dos usuários e dos stakeholders.

5.2 Conceitos de negócios para a engenharia de software

Na engenharia de software, há vários conceitos de negócios fundamentais que são essenciais para entender e incorporar no desenvolvimento de sistemas de software que atendam efetivamente às necessidades das organizações e dos usuários finais. Vamos explorar e reforçar alguns dos principais conceitos de negócios relevantes para a engenharia de software:

- **Objetivos do Negócio**: os objetivos do negócio representam as metas de alto nível que uma organização pretende alcançar por meio do uso de sistemas de software. Eles podem incluir aumentar a eficiência operacional, melhorar a experiência do cliente, reduzir custos, expandir o mercado, entre outros.
- **Processos de Negócio**: os processos de negócio são as atividades e fluxos de trabalho que uma organização realiza para alcançar seus objetivos de negócio. Eles descrevem como as tarefas são realizadas, quem são os participantes envolvidos e quais são as regras e procedimentos a serem seguidos.
- **Requisitos de Negócio**: os requisitos de negócio são as necessidades e expectativas específicas das partes interessadas da organização em relação aos sistemas de software. Eles incluem requisitos funcionais e não funcionais que devem ser atendidos para que o sistema seja considerado bem-sucedido.
- **Valor de Negócio**: o valor de negócio representa o benefício ou retorno que uma organização espera obter ao investir em sistemas de software. Isso pode incluir aumento de receita, redução de custos, melhoria da produtividade, vantagem competitiva, entre outros.
- **Modelagem de Negócio**: a modelagem de negócio é o processo de representar e analisar os processos, estruturas, fluxos de trabalho e interações de uma organização por meio de diagramas e modelos. Isso inclui a identificação de stakeholders, a definição de requisitos de negócio e a criação de diagramas de processos de negócio.

- **Arquitetura Empresarial**: a arquitetura empresarial é uma abordagem holística para projetar e gerenciar a estrutura e operações de uma organização, incluindo pessoas, processos, tecnologia e informação. Isso inclui a definição de metas estratégicas, a criação de modelos de negócio e a elaboração de planos de transformação organizacional.
- **Gestão de Mudanças**: a gestão de mudanças é o processo de planejar, implementar e gerenciar mudanças em uma organização, incluindo a adoção de novos sistemas de software. Isso envolve comunicar efetivamente as mudanças, envolver as partes interessadas, mitigar riscos e garantir uma transição suave.

Esses são apenas alguns dos principais conceitos de negócios que são importantes na engenharia de software. Compreender esses conceitos e integrá-los ao processo de desenvolvimento de software é essencial para garantir que os sistemas desenvolvidos atendam efetivamente às necessidades e objetivos das organizações.

FINALIZANDO

Nesta abordagem, tratamos dos conceitos essenciais do nosso estudo. O mais importante é entender que quando falamos de modelagem de processos de negócios estamos falando do início do processo de automação de um software. Ou seja, estamos entendo sobre o negócio para projetar o melhor software possível para resolver o problema do nosso cliente, ou melhor, para agregar valor e competitividade ao negócio.

Para que o entendimento sobre o processo de negócio seja o mais fiel possível com a necessidade do cliente, é preciso que o fluxo esteja detalhado e de acordo com a visão de todos os usuários envolvidos no processo. Isso garante que o fluxo estará completo e adequado ao resultado que se deseja alcançar.

Na modelagem de processos de negócio, é fundamental utilizar um padrão para representar o fluxo de tarefas e acontecimento. A padronização na modelagem ajuda a compreender o funcionamento e a comunicar a todos os envolvidos como o fluxo de processo está estruturado e organizado. Afinal, o grande objetivo de uma modelagem de processos de negócios é deixar claro o fluxo de negócio, permitindo analisar pontos de falha ou de gargalo, que gerem

atrasos no processamento. A modelagem ajuda a compreender o funcionamento dos processos de negócio de qualquer empresa. Consolidando o input que dispara cada processo e as tarefas que são realizadas para gerar um resultado. Logo, todo processo precisa transformar entradas em saídas. Toda saída será uma informação, um serviço ou um produto, que foi gerado a partir de um processamento sequenciado e organizado de passos, chamados de *tarefas*.

Dessa forma, é fundamental o levantamento completo e devidamente sequenciado de todas as tarefas envolvidas no processo, pensando na ordem de execução das mesmas, para que o resultado obtido seja o esperado. É importante entender que um processo bem modelado e bem desenhado é o reflexo de um bom resultado gerado, seja esse resultado uma informação, um produto ou até mesmo um serviço.

Se o processo de negócio definido for engessado, complexo ou incompleto, o software que será criado a partir dele não será um software que atenderá às necessidades do cliente.

Em conteúdos posteriores, vamos nos aprofundar em outros assuntos relacionados com a modelagem de processos de negócios para o desenvolvimento de um software.

REFERÊNCIAS

BECKER, J.; ROSEMANN, M.; VON UTHMANN, C. Guidelines of business process modeling. In: VAN DER AALST, W.; DESEL, J.; OBERWEIS, A. (Ed.). **Business Process Management**. New York: Springer Berlin Heidelberg, 2000. p. 30-49.

COHN, M. **User Stories Applied for Agile Software Development**. Assison-Wesley Professional, 2004.

GANE, C.; SARSON, T. **Análise Estruturada de Sistemas**. LTC. 1983.

LAPLANTE, P. A. **Requirements engineering for software and systems**. Boca Raton: CRC Press, 2013.

MALL, R. **Fundamentals of software engineering**. New Delhi: PHI Learning, 2014.

OGUNNAIKE, B. A.; RAY, W. H. **Process dynamics, modeling, and control**. Oxford: Oxford University Press, 1994.

PAIM, R. et al. **Gestão de Processos**: pensar, agir e aprender. Porto Alegre: Bookman. 2009.

PIZZA, W. R. **A metodologia Business Process Management (BPM) e sua importância para as organizações**. Monografia (Curso de Tecnologia em Processamento de Dados) – Faculdade de Tecnologia de São Paulo, FATEC SP, São Paulo, 2012.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: uma Abordagem Profissional. 8. ed. Mc Graw Hill Educattion, 2016.

ROSEMANN, M.; BROCKE, J. V.; HONORATO, B. **Manual de BPM**: gestão de processos de negócio. Porto Alegre: Bookman, 2013.

WIEGERS, K.; BEATTY, J. **Software requirements**. 3. ed. London: Pearson Education, 2013.